Gridworld game for this book, and it is included in the GitHub repository for this chapter.

3.2.6. Introducing the Gridworld game engine

In the GitHub repository for this chapter, you'll find a file called Gridworld.py. Copy and paste this file into whatever folder you'll be working out of. You can include it in your Python session by running `from Gridworld import *`. The Gridworld module contains some classes and helper functions to run a Gridworld game instance. To create a Gridworld game instance, run the code in the following listing.

Listing 3.1. Creating a Gridworld game

```
from Gridworld import Gridworld
game = Gridworld(size=4, mode='static')
```

The Gridworld board is always square, so the size refers to one side's dimension—in this case a 4 × 4 grid will be created. There are three ways to initialize the board. The first is to initialize it statically, as in listing 3.1, so that the objects on the board are initialized at the same predetermined locations. Second, you can set `mode='player'` so that just the player is initialized at a random position on the board. Last, you can initialize it so that all the objects are placed randomly (which is harder for the algorithm to learn) using `mode='random'`. We'll use all three options eventually.

Now that we've created the game, let's play it. Call the `display` method to display the board and the `makeMove` method to make a move. Moves are encoded with a single letter: *u* for up, *l* for left, and so on. After each move, you should display the board to see the effect. Additionally, after each move you'll want to observe the reward/outcome of the move by calling the `reward` method. In Gridworld, every nonwinning move receives a –1 reward. The winning move (reaching the goal) receives a +10 reward, and there's a –10 reward for the losing move (landing on the pit).

```
>>> game.display()
array([['+', '-', ' ', 'P'],
       [' ', 'W', ' ', ' '],
       [' ', ' ', ' ', ' '],
```